

Exercise 2: E-commerce Platform Search Function

1. Aim

To implement **Linear Search** and **Binary Search** algorithms in Java for searching products in an e-commerce platform and compare their performance using **Big O notation**.

2. Problem Understanding

In an e-commerce platform, users search for products frequently. As the number of products increases, the search algorithm must be efficient to provide quick results.

Algorithms help improve the performance of searching operations by reducing the time required to locate a product.

3. Algorithm

Linear Search

1. Start from the first element.
2. Compare each product ID with the search key.
3. If found, return the product.
4. If the end of the array is reached, return "Product Not Found".

Binary Search

1. Sort the array by Product ID.
2. Find the middle element.
3. Compare the middle element with the search key.
4. If equal, return the product.
5. If the key is smaller, search the left half.
6. Otherwise, search the right half.
7. Repeat until the product is found or the search space becomes empty.

Search Operation Cases

Best Case:

- The element is found immediately.
- Example: The first element matches the search key.

Average Case:

- The element is found somewhere in the middle of the data.

Worst Case:

- The element is the last element or is not present.
- The algorithm checks every element before concluding.

4.Programming/Output:

```
J Exercise2.java > Exercise2
1  import java.util.Arrays;
2  import java.util.Comparator;
3  class SearchProduct{
4      int productId;
5      String ProductName;
6      String Category;
7      SearchProduct(int productId,String ProductName,String Category){
8          this.productId = productId;
9          this.ProductName = ProductName;
10         this.Category = Category;
11     }
12
13     @Override
14     public String toString(){
15         return "Product ID: "+productId+", Name: "+ProductName+", Category: "+Category;
16     }
17 }
18 public class Exercise2 {
19     public static SearchProduct linearSearch(SearchProduct[] products, int id){
20         for(SearchProduct product: products)
21         {
22             if(product.productId==id){
23                 return product;
24             }
25         }
26         return null;
27     }
28     public static SearchProduct BinarySearch(SearchProduct[] products, int id){
29         int low = 0;
30         int high = products.length-1;
31         while(low<=high){
32             int mid = (low+high)/2;
33             if(products[mid].productId==id){
34                 return products[mid];
35             }
36             else if(products[mid].productId<id){
37                 low = mid+1;
38             }
39             else{
40                 high= mid-1;
41             }
42         }
43         return null;
44     }
45 }
```

```
46     public static void main(String[] args) {
47         SearchProduct[] products={
48             new SearchProduct(productId: 101, ProductName: "Keyboard", Category: "Electronics"),
49             new SearchProduct(productId: 105, ProductName: "Laptop", Category: "Electronics"),
50             new SearchProduct(productId: 103, ProductName: "Shoes", Category: "Fashion"),
51             new SearchProduct(productId: 104, ProductName: "Mobile", Category: "Electronics"),
52             new SearchProduct(productId: 102, ProductName: "Watch", Category: "Accessories"),
53         };
54         int searchId = 104;
55         System.out.println(x: "Linear Search");
56         SearchProduct result1 = linearSearch(products,searchId);
57         if(result1!=null){
58             System.out.println(result1);
59         }
60         else{
61             System.out.println(x: "Product not Found");
62         }
63         System.out.println();
64         Arrays.sort(products,Comparator.comparingInt(p->p.productId));
65         System.out.println(x: "Binary Search");
66
67         SearchProduct result2 = BinarySearch(products, searchId);
68         if(result2!=null)
69             System.out.println(result2);
70         else
71             System.out.println(x: "Product Not Found");
72     }
73 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\Krishnavishwa\OneDrive\Documents\Material\cognizant\Algorithm> & 'C:\Program Files\Java\jdk-17\bin\java.exe' -cp 'C:\Program Files\Java\jdk-17\bin\java.exe;C:\Program Files\Java\jdk-17\bin\redhat.java\jdt_ws\Algorithm_cc0d2e2b\bin' 'Exercise2'

Linear Search
Product ID: 104, Name: Mobile, Category: Electronics

Binary Search
Product ID: 104, Name: Mobile, Category: Electronics

5. Time Complexity Analysis

Operation	Best Case	Average Case	Worst Case
Linear Search	O(1)	O(n)	O(n)
Binary Search	O(1)	O(log n)	O(log n)

6. Comparison

Linear Search	Binary Search
Works on unsorted data	Requires sorted data
Simple to implement	Slightly more complex
Slower for large datasets	Faster for large datasets
Time Complexity: $O(n)$	Time Complexity: $O(\log n)$

7.Result

Thus, the **Linear Search** and **Binary Search** algorithms were successfully implemented and compared. Linear Search is simple and works on unsorted data but becomes inefficient for large datasets. Binary Search provides much better performance for sorted data with a time complexity of **$O(\log n)$** , making it the preferred choice for implementing search functionality in an e-commerce platform.